

LTS-RK4 multi-niveaux pour le maillage gradué en L

Journal d’implémentation et de validation

Romain Mottier

23 août 2026

Résumé

Ce rapport documente la conception, l’implémentation et la validation d’une généralisation multi-niveaux du schéma explicite *Local Time-Stepping* RK4 (LTS-RK4, “Algorithme 3”) déjà présent dans DiSk++ (`erk_coupling_hho_scheme`), motivée par le coût prohibitif du schéma à 2 niveaux existant sur un maillage gradué en L avec singularité de coin rentrant. Le rapport couvre : le diagnostic du problème de coût, deux bugs de correction trouvés dans le chemin de code “restreint” préexistant, les nouvelles briques de calcul restreintes et validées, la généralisation récursive multi-niveaux, et l’état actuel de validation.

Table des matières

1	Contexte et motivation	1
1.1	Le problème de coût	1
1.2	L’observation de Grote	1
1.3	Diagnostic : distribution des tailles de cellules	2
2	Diagnostic empirique de deux bugs préexistants	2
3	Nouvelles briques restreintes (fichier <code>erk_coupling_hho_scheme.hpp</code>)	3
3.1	LTS_subblock_set et build_LTS_subblock	3
3.2	Rôle “coarse” : <code>erk_weight_LTS_coarse_v2</code>	3
3.3	Rôle “fine” : <code>erk_weight_LTS_fine_v2</code>	3
3.4	Repli analytique : <code>erk_weight_LTS_coarse_advance_uncovered</code>	3
4	Validation à $L = 2$	4
5	Généralisation multi-niveaux	4
5.1	Structure récursive	4
5.2	Point clé : mise à l’échelle du pas de temps par niveau	5
5.3	Validation de la récursion générique	5
5.4	Résultats à $L = 5$ ($N = 2$, $k = 3$)	5
6	État actuel et travail restant	5

1 Contexte et motivation

1.1 Le problème de coût

Le maillage en L gradué vers le coin rentrant (angle intérieur $\omega = 3\pi/2$) est construit par un raffinement quadtree couplé : le fond est raffiné uniformément ℓ fois, et le coin est raffiné avec une profondeur supplémentaire $\text{depth} = 1.5(k+1)\ell$ (pour un degré polynomial HHO k), afin de retrouver le taux de convergence optimal h^{k+1} malgré la singularité (théorie classique du maillage gradué).

Cette construction fait exploser le ratio $p = h_{\max}/h_{\min}$: $p = 1, 32, 1024, 32768, \dots$ pour les niveaux successifs $N = 0, 1, 2, 3, \dots$ (pour $k = 3$, p croît d’un facteur 64 par niveau). Le schéma LTS-RK4 à 2 niveaux existant effectue p sous-pas RK4 fins par macro-pas, et — point crucial — **chaque appel touche l’intégralité du maillage**, pas seulement la région “fine”. Le coût total est donc $O(p \times n_{\text{cells}})$, ce qui rend $N = 3$ ($p \approx 32768$) de l’ordre de 9 à 10 heures de calcul, et $N = 4$ totalement hors de portée.

1.2 L’observation de Grote

Un schéma LTS-RK4 correctement implémenté devrait coûter approximativement la même chose qu’un RK4 classique (sans découpage coarse/fine) dès lors que la région fine est minoritaire en nombre de degrés de liberté — c’est précisément ce qui n’était pas observé. Ceci a motivé une investigation en deux temps :

1. rendre le schéma à 2 niveaux *effectivement* restreint en coût (chaque appel ne doit toucher que les DOFs de la bande active, pas tout le maillage) ;
2. généraliser à un véritable schéma multi-niveaux, nécessaire car aucun seuil unique ne peut séparer “fine minoritaire” de “coarse stable au grand pas de temps” sur un maillage dont la distribution de tailles de cellules est quasi uniforme en échelle logarithmique sur 10 à 15 octaves (vérifié empiriquement, voir §1.3).

1.3 Diagnostic : distribution des tailles de cellules

Un histogramme des diamètres de cellules par octave ($h_{\max}/2^k$) révèle que le fond uniformément raffiné (bande 0) est la seule bande franchement majoritaire ; chaque octave suivante, jusqu’à 15 octaves de profondeur à $N = 3$, ne contient qu’un nombre quasi constant de cellules (~ 36), conséquence directe de la marge de raffinement (“halo”) ajoutée autour du coin. Cela prouve qu’**aucun seuil unique** ne peut donner la propriété “fine minoritaire” de Grote sans que la bande opposée (“coarse”) ne s’étende elle-même sur de nombreuses octaves — ce qui, comme démontré en §5.2, casse la validité de l’approximation polynomiale du rôle coarse.

2 Diagnostic empirique de deux bugs préexistants

Avant de construire quoi que ce soit de nouveau, le chemin de code “restreint” déjà présent dans `erk_coupling_hho_scheme.hpp` (`build_LTS_subspaces + erk_weight_LTS_coarse_restricted / erk_weight_LTS_fine_restricted`, utilisé par `ERK4_LTS_optimised.hpp`) a été audité par lecture de code, puis **confirmé empiriquement défaillant** par un test contrôlé : sur un maillage localement raffiné (ratio $p = 2$), avec un pas de temps correctement résolu (CFL vérifié via une comparaison avec $p = 1$, erreur ~ 0.02 – 0.10 , plausible), le chemin restreint donne une erreur 10

à 30 fois *pire* malgré un maillage plus fin — signature claire d’un bug de correction, pas d’un problème de précision.

Bug A — dofs coarse jamais avancés. Dans `erk_weight_LTS_fine_restricted`, la mise à jour RK4

$$\delta x = d\tau \cdot \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

n’est diffusée (*scatter*) que sur les positions *fin*es (`m_fine_active_c/f`) ; un commentaire affirme que les dofs coarses sont “gérés ailleurs”, mais `erk_weight_LTS_coarse_restricted` ne fait *jamais* muter l’état — elle ne fait qu’écrire dans `w[]`. Aucune autre voie de code ne met à jour les dofs coarses : ils restent gelés à leur valeur initiale pendant toute la simulation.

Bug B — terme de couplage à l’interface manquant. Dans `compute_w_restricted`, la sortie “face” n’est calculée et diffusée que pour les faces *classées coarses*. Or une face d’interface (un voisin coarse, un voisin fine) est *toujours* classée fine par la règle d’union de `assemble_P` (“une face est fine si au moins une cellule adjacente est fine”). La contribution de la cellule coarse vers cette face d’interface — pourtant non nulle et physiquement nécessaire au couplage — est donc silencieusement perdue.

Ces deux bugs affectent uniquement `ERK4_LTS_optimised.hpp` et `ERK4_LTS_SSTAB.hpp` ; ils sont **indépendants** des prototypes L-shape de cette session, qui utilisent l’API non restreinte (`erk_weight_LTS_coarse/erk_weight_LTS_fine`) et restent donc fiables. Par principe de moindre risque, ces fonctions préexistantes n’ont *pas* été modifiées ; de nouvelles fonctions, correctement conçues dès le départ, ont été ajoutées à la place (voir §3).

3 Nouvelles briques restreintes (fichier `erk_coupling_hho_scheme.hpp`)

3.1 `LTS_subblock_set` et `build_LTS_subblock`

Contrairement à `build_LTS_subspaces` (un seul jeu de sous-blocs coarse/fine, dans des membres de classe uniques, écrasés à chaque appel), `build_LTS_subblock(own_c, own_f, halo_rings)` retourne une structure *autonome* `LTS_subblock_set`, permettant à plusieurs bandes de coexister — prérequis indispensable pour le multi-niveaux.

Le halo est construit par parcours en largeur (BFS) sur le graphe d’adjacence cellule–face induit par la structure creuse de K_{fc} : partant de `own_c/own_f`, chaque anneau ajoute les faces touchant une cellule déjà incluse, puis les cellules touchant une face déjà incluse. Ce halo garantit que la chaîne d’applications répétées de l’opérateur B (utilisée par le rôle “coarse”, voir §3.2) reste exacte, en exploitant le fait que K_{cc} est exactement diagonale par bloc par cellule en HHO (les cellules ne couplent qu’à travers les faces).

Chaque `LTS_subblock_set` stocke *deux* jeux de sous-matrices restreintes : $K_{cc}/K_{cf}/K_{fc}/M_{c_inv}/S_{ff_inv}$ (avec halo, pour le rôle coarse) et leurs équivalents `_own` (sans halo, pour le rôle fine, qui n’en a pas besoin — voir §3.3).

3.2 Rôle “coarse” : `erk_weight_LTS_coarse_v2`

Reproduit fidèlement, mais restreint à `blocks.active_c/f` (propre + halo), l’algorithme de `erk_weight_LTS_coarse` : interpolation de Lagrange quadratique de la force sur $[0, dt]$, chaînes $B^i y_n$ et $B^i F_j$ (non masquées, utilisant le halo pour rester exactes), puis masquage *final* à `own_c/own_f` uniquement avant la dernière application de B — ce masquage final est ce qui donne au polynôme

de Taylor de sortie $w[0..3]$ sa fuite correcte vers les faces d’interface (fixant le Bug B), sans polluer les cellules halo elles- mêmes (K_{cc} diagonale par cellule + masquage $\Rightarrow$ contribution nulle aux lignes non-propres).

Sortie : $w_i(\tau) = w_0 + \tau w_1 + \frac{\tau^2}{2} w_2 + \frac{\tau^3}{6} w_3$, valide en temps local sur $[0, \text{len}]$ où len est la durée *propre* à ce niveau (voir §5, pas le macro-pas global).

3.3 Rôle “fine” : `erk_weight_LTS_fine_v2`

Un pas RK4 complet de taille $d\tau$, avec chaque étage $k = B(P_{\text{own}}y) + w(\tau)$ — l’entrée y est masquée à `own` avant application de B (miroir exact de $P_{\text{fine}} \cdot y_{\text{stage}}$ dans l’algorithme original), *mais* B est appliqué avec les matrices *avec halo*, de sorte que la sortie soit non nulle non seulement sur les positions propres, mais aussi sur les cellules coarse directement adjacentes (halo) — exactement le comportement de l’algorithme original non restreint, où une cellule coarse au bord hérite d’une contribution réelle via le couplage K_{cf} à la face d’interface. La mise à jour est diffusée sur `active_c/f` en entier (propre + halo), corrigeant le Bug A.

3.4 Repli analytique : `erk_weight_LTS_coarse_advance_uncovered`

Les cellules coarse *non* atteintes par le halo de la bande fine (les cellules “profondément intérieures”) ne reçoivent aucune correction du rôle fine. Pour elles, $B(P_{\text{fine}}y) \equiv 0$ (aucun couplage vers une face fine), donc leur mise à jour exacte sur tout le macro-pas est l’intégrale fermée du polynôme cubique :

$$\Delta x = w_0 dt + w_1 \frac{dt^2}{2} + w_2 \frac{dt^3}{6} + w_3 \frac{dt^4}{24}.$$

Ceci est *exact*, pas une approximation : la règle de Simpson (à laquelle se réduit la somme RK4 lorsque $B(P_{\text{fine}}y) = 0$ identiquement) est exacte pour un polynôme cubique. **Cellules couvertes par le halo et cellules non couvertes ne doivent jamais recevoir les deux mécanismes** — ce fut la source d’un bug de double comptage (et d’un bug de comptage manquant dans une première tentative), corrigé et validé (§4).

4 Validation à $L = 2$

Méthodologie : comparaison directe, pas à pas, du vecteur d’état complet entre l’algorithme original (non restreint, éprouvé) et les nouvelles briques v2, sur un maillage réel avec un partage coarse/fine *authentique* (pas un cas dégénéré où l’une des deux bandes est vide). Deux bugs ont été trouvés et corrigés durant cette validation avant d’obtenir une correspondance exacte :

1. Une première tentative (`fine_v2` n’écrivant que ses positions propres, complétée par une intégrale fermée *inconditionnelle* pour *toutes* les cellules coarse propres) donnait un écart de 1.3×10^{-5} après un pas, croissant à 0.05 après 141 pas — cause : les cellules coarses *au bord* recevaient alors À LA FOIS la correction $B(P_{\text{fine}}y)$ (implicitement, puisque non masquée dans cette version) ET l’intégrale fermée, un double comptage partiel.
2. Une seconde tentative (halo inclus dans `fine_v2`, diffusion sur `active_c/f` complet, mais *sans* intégrale fermée du tout) donnait un écart *pire* (10^{-3} après un pas) — cause : les cellules coarses non atteintes par le halo (environ 72/864 dofs dans le cas testé) ne recevaient alors *aucune* mise à jour du tout.

3. La combinaison correcte — intégrale fermée appliquée *seulement* aux cellules propres de la bande coarse non couvertes par le halo de la bande fine (vérifié par appartenance d'ensemble) — donne une correspondance exacte à la précision machine (9.7×10^{-16} , arrondi) sur les 141 pas de temps complets.

5 Généralisation multi-niveaux

5.1 Structure récursive

À chaque niveau $i < L - 1$: calculer w_i (rôle coarse, avec la durée *propre* à ce niveau len_i , pas le macro-pas global), combiner avec la contribution ancêtre reçue en argument (somme coefficient par coefficient, les deux étant exprimés dans le même repère temporel local), avancer les cellules propres non couvertes via l'intégrale fermée du polynôme *combiné*, puis boucler p_i fois : à chaque sous-intervalle, décaler (*Taylor-shift*) le polynôme combiné vers la nouvelle origine locale, et soit appeler `erk_weight_LTS_fine_v2` directement (cas de base, $i + 1 = L - 1$), soit récurser.

Décalage de Taylor. Pour un polynôme cubique $T(\tau) = w_0 + w_1\tau + \frac{w_2}{2}\tau^2 + \frac{w_3}{6}\tau^3$, la ré-expansion exacte autour d'un nouveau point a est :

$$w'_0 = T(a), \quad w'_1 = T'(a), \quad w'_2 = T''(a), \quad w'_3 = w_3.$$

Ceci est une identité exacte (série de Taylor finie d'un polynôme de degré 3), pas une approximation.

5.2 Point clé : mise à l'échelle du pas de temps par niveau

Une première tentative de §5 appliquée à un simple split à 2 niveaux, en réduisant agressivement le seuil h_c (pour minimiser la région “fine”) *tout en gardant le même pas de temps macro* $dt = \text{cfl} \cdot h_{\max}$, a produit un résultat NaN — même avec $h_c = 16 h_{\min}$ (bande coarse s'étalant encore sur ~ 6 octaves). Diagnostic : le polynôme cubique $w[]$ n'a que 4 degrés de liberté ; sur une fenêtre dt calée pour des cellules de taille $h_{\max}$, il ne peut pas représenter la dynamique de cellules bien plus petites oscillant plusieurs fois pendant cette même fenêtre — rupture de validité inhérente à la méthode, pas un bug d'implémentation (les coefficients eux-mêmes restent corrects, comme vérifié en §4).

Correction : chaque niveau i reçoit son propre $dt_i = \text{cfl} \cdot \text{scale}_i$, où scale_i est la taille caractéristique des cellules de ce niveau ($h_{\max}$ pour le niveau 0, le seuil $h_{c,i}$ pour les niveaux intermédiaires, $h_{\min}$ pour le niveau le plus fin). Le ratio de branchement est $p_i = \text{round}(dt_i/dt_{i+1})$.

5.3 Validation de la récursion générique

Le pilote récursif générique, spécialisé à $L = 2$ avec exactement le même seuil que la validation de §4, reproduit le résultat de référence à la précision machine (6.9×10^{-16}). Ceci confirme que la récursion elle-même (décalage de Taylor, accumulation des contributions ancêtres, repli analytique) est correcte — tout écart constaté à $L > 2$ est donc une question de réglage (précision), pas de correction.

5.4 Résultats à $L = 5$ ($N = 2$, $k = 3$)

Avec des bandes espacées d'environ 2 octaves chacune (5 niveaux au total pour couvrir les 10 octaves de $p = 1024$) :

- Aucune divergence (NaN) — confirmation de la correction de mise à l'échelle du pas de temps.
- Temps de calcul : 170 s, contre ~ 540 s pour le schéma original non restreint — gain $\times 3,2$.
- Précision : erreur $L^2 = 3,7 \times 10^{-3}$, contre $1,14 \times 10^{-5}$ pour le schéma de référence — une dégradation significative (facteur ~ 326), attribuable à l'accumulation d'approximations cubiques à travers 4 niveaux intermédiaires plutôt qu'à une erreur de conception.

6 État actuel et travail restant

- **Acquis et validé** : briques restreintes (coarse/fine/repli), pilote récursif générique — tous deux exacts par rapport à l'algorithme de référence dans leur cas de test respectif.
- **Acquis mais à régler** : le schéma multi-niveaux complet est stable et plus rapide, mais nécessite un réglage (octaves par niveau, `cfl_factor`) pour retrouver la précision de référence sans perdre le gain de vitesse.
- **Prochaine étape** : étude paramétrique du compromis précision/vitesse, puis application à $N = 3$ et $N = 4$ pour compléter la courbe de convergence à 5 points.